

Table of Contents

PRD — n8n Execution Analyst

Project codename: n8n-execution-analyst **Author:** Patrick Diamitani — GTM AI & Automation Manager, Atlas HXM **Date:** 2026-06-16 **Status:** Draft v1.0 (for review)

Methods applied: ROSTR (development + end-user integration), Jobs To Be Done (JTBD), n8n Public API best practices

0. Document Map (the 42 required sections)

This PRD answers every section Patrick specified. For navigation:

1. PRD (this document) · 2. Tech Stack · 3. Production · 4. Development · 5. Key Sheet · 6. Secret Keys · 7. Logins · 8. Data Resources · 9. End Users · 10. Overview · 11. Features · 12. Deployment Plan · 13. Build Plan · 14. Testing Plan · 15. Tools & Scripts · 16. Documentation Needed · 17. Next Steps · 18. Milestones · 19. Project Goals (KPIs & Reporting) · 20. Business Use Case · 21. Collaborators · 22. Owner · 23. Manager · 24. Team/Builders · 25. ELT/Function · 26. Project KPIs & Reporting Framework · 27. Reporting Platforms · 28. Source · 29. Display · 30. KPI Type · 31. Daily/Weekly/Monthly Metrics · 32. # Runs/Executions · 33. # Users · 34. # Requests/Tickets · 35. # Reports · 36. Other Metrics · 37. Dashboard Architecture · 38. Output Surfaces (Excel/HubSpot/HTML) · 39. Milestones, Wins & Goals · 40. Timelines · 41. P&L Ownership · 42. Credit/Fault Accountability.

Two methodology sections (JTBD, ROSTR) precede the 42 to anchor the design.

A. Jobs To Be Done (JTBD) Anchor

Main Job Statement

When a GTM automation in our n8n instance runs (or fails to run), I want to know *what happened, where, and why* — across every workflow, every day, without manually opening each execution — so that I can keep our pipeline automations reliable and prove their business impact.

Functional Jobs

- Capture every execution (live + inactive workflows) with full node-level detail.
- Pinpoint the exact node where a run stopped and the reason.
- Store execution history durably so trends are queryable over time.

- Answer natural-language questions about runs, dates, failures, and reliability.
- Alert the AI GTM team the moment something breaks or stalls.

Emotional Jobs

- Confidence that automations are healthy (no silent failures eroding pipeline).
- Relief from the anxiety of “is something broken right now that I don’t know about?”
- Credibility with ELT — showing automation reliability and ROI with hard data.

Triggering Context

Daily standup readiness; a rep reports “the enrichment didn’t run”; an ELT review needs uptime/ROI numbers; a workflow silently fails over a weekend.

Importance vs. Satisfaction (current state)

Outcome	Importance (1–10)	Current Satisfaction (1–10)	Opportunity Gap
Know immediately when a workflow fails	10	3	HIGH
See exactly which node failed and why	9	2	HIGH
Query historical execution trends	8	1	HIGH
Prove automation ROI to ELT	9	3	HIGH
Daily health summary without manual checks	8	2	HIGH

Conclusion: All five top outcomes are high-importance / low-satisfaction → this is a high-value, under-served job. Build is justified.

Job vs. Solution

- **The job** is reliable visibility + accountability into automation health.
- **The solution category** chosen: a **Claude skill + scheduled ingestion pipeline + queryable store + dashboard + alerting**. (Not “a prettier n8n UI” — n8n already shows single executions; the unmet job is *aggregation, history, NLP query, and proactive alerting*.)

B. ROSTR Method Mapping (development + product)

ROSTR is applied as the build discipline. Each layer is mapped here; the rostr-builder skill should be run during Build Plan Step 0 to formalize the declarative spec.

- **NPAO task canvas (N→A→P→O):** classify every capability as **Navigate** (find workflows/executions), **Act** (ingest, write to store, send alerts), **Process** (parse JSON, diagnose failure node), **Output** (dashboard, NL answer, report).
 - **4Ds phases:** **Discover** (API recon, schema design) → **Design** (skill arch, dashboard) → **Develop** (scripts, ingestion, query engine) → **Deploy** (schedule, alerting, docs).
 - **ROSTR HUB layers:** Retrieval (n8n API), Orchestration (ingestion + scheduler), Storage (SQLite→Postgres), Transformation (parse/diagnose), Reporting (dashboard/NLP).
 - **RAG / DAL:** the NLP query layer retrieves from the structured store (Data Access Layer) — no free-text hallucination; answers cite execution IDs and dates.
 - **ContextEngine memory:** persist run-state and known-issue patterns so recurring failures are recognized across sessions.
-

1. PRD

This document. It defines scope, architecture, build plan, testing, KPIs, and ownership for the n8n Execution Analyst. It is the single source of truth and is version-controlled in the project folder.

2. Tech Stack

Layer	Choice	Rationale
Source API	n8n Public REST API v1	Native, stable, full execution + node data
Ingestion runtime	Python 3.11 (requests, pydantic)	Portable, easy parsing of nested JSON
Data store (Phase 1)	SQLite	Zero infra, single-file, portable
Data store (Phase 2)	PostgreSQL	Multi-user, concurrent, scales with run volume
Scheduler	Cowork scheduled task (daily) + on-demand skill invocation	No extra infra for v1
Query/NLP	Claude skill (this) over the DAL	Natural-language analytics, grounded
Dashboard	Self-contained HTML (Chart.js + Grid.js via CDN)	Opens anywhere, live-refresh via artifact
Alerting	Slack/Teams webhook + email	Ping AI GTM team on failure/stall
Packaging	Claude Skill (SKILL.md + /scripts)	Reusable by the team, at-will + scheduled

3. Production

- **Environment:** Atlas n8n Cloud — <https://atlas-hxm.app.n8n.cloud> (instance confirmed reachable).
- **Prod data store:** Postgres (Phase 2) on Atlas-managed infra; SQLite file in the project folder for Phase 1 pilot.
- **Prod cadence:** daily scheduled ingestion at 06:00 CT + real-time at-will runs.
- **Prod guardrails:** read-only API scope where possible; no destructive calls (DELETE /executions is **disabled** in skill code); secrets via env vars, never in code.

4. Development

- **Dev environment:** Cowork sandbox (Python preinstalled) + project folder for code/artifacts.
- **Dev data store:** local SQLite seeded from a small limit pull (≤ 50 executions) to protect rate limits.
- **Branching:** PRD → build guide → scaffolding → iterative scripts. Each script independently testable.
- **Coding standards:** modular (ingest / store / diagnose / query / report as separate modules), typed, no hardcoded secrets, idempotent ingestion (upsert on execution ID).

5. Key Sheet

Item	Value / Location	Notes
n8n instance URL	https://atlas-hxm.app.n8n.cloud	Confirmed live
n8n API base	https://atlas-hxm.app.n8n.cloud/api/v1	Public API v1
Auth header	X-N8N-API-KEY: <key>	Provided key returned 401 — full JWT key still needed
Executions endpoint	GET /executions?includedata=true&limit=250&cursor=...	Paginated
Single execution	GET /executions/{id}?includeData=true	Full node runData
Workflows endpoint	GET /workflows?limit=250&cursor=...	Names + active flag
Single workflow	GET /workflows/{id}	Node definitions
Store (Phase 1)	./data/executions.sqlite	Project folder
Dashboard	./dashboard/index.html	Self-contained

6. Secret Keys

- **n8n API key** — the only secret required for v1. Stored in an environment variable `N8N_API_KEY` (never committed).
- **ACTION REQUIRED:** the pasted key `d2f7d5ee6818f9454583` is invalid (401, too short for an n8n JWT). Provide the full key from n8n → **Settings** → **n8n API** → **Create an API key**. Rotate the pasted key for hygiene.
- Alert webhook URLs (Slack/Teams) — stored as `ALERT_WEBHOOK_URL` env var when alerting is wired.
- No secrets are written to the data store, logs, or the dashboard.

7. Logins

System	Access needed	Who holds it
n8n Cloud	Owner/admin to mint API key	Patrick
Slack/Teams (alerts)	Incoming webhook permission	Patrick / IT
Postgres (Phase 2)	DB credentials	Patrick / infra owner
Cowork / project folder	Already authenticated	Patrick

8. Data Resources (Links)

- n8n instance: <https://atlas-hxm.app.n8n.cloud>
- n8n Public API reference: <https://docs.n8n.io/api/>
- n8n API auth: <https://docs.n8n.io/api/authentication/>
- n8n pagination: <https://docs.n8n.io/api/pagination/>
- Project folder (deliverables + store): Atlas Skill Builder - Best Practices/

9. End Users

- **Primary:** Patrick (GTM AI & Automation) — daily health checks, ad-hoc investigations.
- **Secondary:** AI GTM team / RevOps — receive failure alerts, run NLP queries.
- **Tertiary:** ELT — consume the dashboard and ROI/uptime reports.
- **Design constraint:** secondary/tertiary users are **non-technical** — the skill must answer plain-English questions and the dashboard must be self-explanatory.

10. Overview

The n8n Execution Analyst is a Claude skill plus automation that continuously (daily) and on-demand pulls execution logs for **all** workflows (active and inactive) from Atlas's n8n instance, extracts complete node-level detail (full JSON, the stop-node and reason, per-node errors), stores every run in a durable, queryable schema (one workflow → many dated execution sub-records), renders an organized HTML dashboard, answers natural-language analytics questions, and proactively pings the AI GTM team when a workflow fails or stalls. It does not invent data — it extracts and reports only what the API returns.

11. Features

1. **Full inventory pull** — every workflow (active + inactive) with ID and name.
2. **Execution capture** — per execution: workflow name + ID, execution date/time, status, mode.
3. **Node-level extraction** — full JSON for each node's execution data (attached/linked/inline text).
4. **Flow analysis** — last node executed + where/why the run stopped.
5. **Error reporting** — per-node error message, stack, and failing node identity.
6. **Master store** — SQLite→Postgres; workflow table + dated execution sub-records.
7. **Dashboard** — HTML data table/visuals: runs, success/fail, recurring problems.
8. **NLP + semantic query** — answer questions by ID, date, run count, success rate, recurring failures.
9. **Daily run tracking** — successes + failures; detect stalls/stops.
10. **Proactive alerting** — ping AI GTM team on failure/stall.
11. **At-will + scheduled** — run daily automatically or any time on request.
12. **No-hallucination guarantee** — answers grounded in stored records, citing execution IDs/dates.

12. Deployment Plan

1. **Pilot (local)**: SQLite store + manual skill runs; validate extraction accuracy on a few workflows.
2. **Scheduled ingestion**: Cowork scheduled task pulls daily at 06:00 CT, idempotent upsert.
3. **Alerting live**: wire Slack/Teams webhook on detected failures/stalls.
4. **Dashboard publish**: HTML artifact, refreshable; optional Azure SWA host for a shareable link.
5. **Scale**: migrate store to Postgres; extend cadence (hourly) if run volume warrants.
6. **Handoff**: skill packaged + docs; team can invoke by name.

13. Build Plan

Detailed in the companion **Build Guide** (n8n-execution-analyst-BUILD-GUIDE.md). High level: Discover (API recon) → Design (schema + skill arch) → Develop (ingest, parse, store, query, dashboard, alert) → Deploy (schedule, docs, handoff). Each phase decomposed into nested sub-tasks with tools, tests, and agent-handoff prompts.

14. Testing Plan

- **Unit**: parser correctly extracts node runData, last-node, errors from sample execution JSON (fixtures captured from live API).
- **Integration**: end-to-end pull → store → query against a known workflow; row counts match API limit.
- **Idempotency**: re-running ingestion does not duplicate execution IDs (upsert verified).
- **Failure-detection**: synthetic failed execution triggers an alert exactly once.

- **NLP accuracy:** a fixed question set (“how many failures yesterday for workflow X?”) checked against ground-truth counts.
- **No-hallucination check:** every NLP answer references real execution IDs present in the store.
- **Verification subagent:** independent pass over diagnosis logic before each release.

15. Tools & Scripts

Script	Purpose
<code>ingest.py</code>	Pull workflows + executions (paginated), upsert to store
<code>parse.py</code>	Extract node runData, last-node, per-node errors from execution JSON
<code>store.py</code>	Schema init + upsert/query DAL (SQLite, Postgres-ready)
<code>diagnose.py</code>	Determine stop-node + reason; classify error types; flag recurring patterns
<code>query.py</code>	NLP/semantic query over the DAL (grounded answers)
<code>alert.py</code>	Send failure/stall pings to Slack/Teams/email
<code>dashboard/build.py</code>	Generate self-contained HTML dashboard
<code>SKILL.md</code>	Skill entry point + routing logic

16. Documentation Needed

- README (setup, API key, run commands).
- Schema doc (tables, columns, relationships).
- Skill usage guide (NLP query examples for non-technical users).
- Runbook (what to do when an alert fires).
- API reference notes (endpoints, params, rate limits).

17. Next Steps

1. Patrick provides the valid n8n API key.
2. Run live API recon to capture real workflow inventory + sample execution JSON (grounds the schema).
3. Approve schema + dashboard mockup.
4. Build scripts iteratively per the Build Guide.
5. Stand up scheduled daily ingestion + alerting.
6. Package skill and hand off to the AI GTM team.

18. Milestones

#	Milestone	Definition of done
M1	API access verified	Valid key returns workflows + executions
M2	Schema live	Store created; sample data ingested
M3	Extraction validated	Node JSON, stop-node, errors correct vs. n8n UI
M4	Dashboard v1	HTML table/visuals render real data
M5	NLP query v1	Plain-English questions answered, grounded
M6	Alerting live	Failure/stall pings AI GTM team
M7	Scheduled + packaged	Daily ingestion runs; skill handed off

19. Project Goals (KPIs & Reporting)

- **Goal 1:** 100% of workflows monitored daily (coverage).
- **Goal 2:** Mean time to detect (MTTD) a failure < 24h (target < 1h once hourly).
- **Goal 3:** Zero silent failures over a rolling 30 days.
- **Goal 4:** Quantify automation reliability % and time saved for ELT.
- Reporting cadence: daily digest (Slack), weekly summary, monthly ELT report.

20. Business Use Case

Atlas's GTM pipeline depends on n8n automations (Clay enrichment, HubSpot sync, Factors.ai signals, Amplemarket enrollment). A silent failure breaks prospecting and costs pipeline. Today, failures are found reactively when a rep notices missing data. This project converts that into proactive, measured reliability — protecting pipeline throughput and giving the AI GTM function hard evidence of uptime and ROI. **Value drivers:** prevented pipeline loss, hours saved on manual log-checking, ELT credibility.

21. Collaborators

- AI GTM team (alert recipients, NLP users)
- RevOps (consumers of reliability data)
- IT/Infra (Postgres, webhooks) — as needed

22. Owner

Patrick Diamitani — product owner, accountable for delivery and outcomes.

23. Manager

Patrick's reporting line (GTM leadership / ELT sponsor) — to be named for the project record.

24. Team / Builders

- **Builder:** Claude (Cowork) under Patrick's direction.
- **Reviewer:** Patrick (+ optional verification subagent).
- **Future:** any GTM engineer onboarded to maintain.

25. ELT / Function

- **Function:** Go-To-Market — AI & Automation.
- **ELT sponsor:** GTM leadership (consumes monthly reliability/ROI report).

26. Project KPIs & Reporting Framework

KPI	Definition	Target	Cadence	Source	Display
Workflow coverage	% workflows ingested daily	100%	Daily	n8n API	Dashboard
Success rate	successful / total executions	$\geq 95\%$	Daily/Weekly	Store	Dashboard
Failure count	# failed executions	trend ↓	Daily	Store	Dashboard + alert
MTTD	time from failure to detection	< 1h (goal)	Per incident	Scheduler logs	Report
Recurring issues	failures with repeated node/error	trend ↓	Weekly	diagnose.py	Dashboard
Time saved	manual log-check hours avoided	report	Monthly	Estimate	ELT report

27. Reporting Platforms

- **Primary:** HTML dashboard (self-contained / artifact).
- **Alerts:** Slack or Teams.
- **Optional sync:** HubSpot (note automation health on related deals/campaigns), Clay (if cross-referencing enrichment runs).

- **Export:** Excel/CSV for ELT.

28. Source

- **Primary source:** n8n Public REST API v1 (/executions, /workflows).
- **Derived source:** the master store (SQLite→Postgres) for all historical/NLP queries.

29. Display

- Data table (Grid.js) — sortable: workflow, run date/time, status, last node, error.
- Charts (Chart.js) — runs over time, success vs. fail, top failing workflows/nodes.
- Drill-down — click an execution to view full node JSON.

30. KPI Type

Type	This project's KPI
Time saved	Manual log-checking hours avoided
Revenue protected	Pipeline preserved by catching failures early
Productivity increased	Faster MTTD, fewer broken automations
Risk reduced	Zero silent failures
Money saved	Avoided rework / lost-lead cost

31. Daily / Weekly / Monthly Metrics

- **Daily:** # executions, # success, # fail, new failures, stalled workflows.
- **Weekly:** success-rate trend, top failing workflows/nodes, recurring issues.
- **Monthly:** reliability %, MTTD, time saved, ROI narrative for ELT.

32. # Runs / Executions

Tracked per workflow and in aggregate, by day. Core metric — each execution is a stored sub-record under its workflow.

33. # Users

Tracked: # of people invoking the skill / receiving alerts / viewing the dashboard. Target adoption: AI GTM team + RevOps.

34. # Requests / Tickets

Tracked: # of NLP queries run, # of alerts fired, # of follow-up investigations. (Optional: open HubSpot/Asana tickets for unresolved failures.)

35. # Reports

Tracked: daily digests sent, weekly summaries, monthly ELT reports generated.

36. Other Metrics

- Avg execution time per workflow (performance drift detection).
- Data freshness (lag between run and ingestion).
- Alert precision (false-positive rate).

37. Dashboard Architecture

- **Data layer:** master store (SQLite→Postgres) via DAL.
- **Build:** dashboard/build.py queries the store and writes a self-contained index.html (Chart.js + Grid.js via CDN, all else inline).
- **Live mode:** Cowork artifact that calls the store/refresh on open (Reload button native).
- **Sections:** KPI header (coverage, success rate, failures today) → runs-over-time chart → success/fail chart → failing-workflows table → execution drill-down.

38. Output Surfaces (Excel / HubSpot / HTML / Artifact)

- **HTML dashboard** — primary, always.
- **Excel/CSV** — export for ELT and offline analysis.
- **HubSpot** — optional note/property on related records when an automation feeding a deal fails.
- **Cowork artifact** — live, re-openable dashboard.

39. Milestones, Wins & Goals

- **Quick win (Week 1):** valid API access + first real execution table rendered.
- **Win (Week 2):** daily digest + first caught failure before a human noticed.
- **Goal (Month 1):** 100% coverage, alerting live, ELT report v1.
- **Stretch:** hourly cadence + Postgres + HubSpot sync.

40. Timelines

Phase	Window	Output
Discover	Day 1–2	API recon, real schema
Design	Day 2–3	Schema + dashboard mockup approved
Develop	Day 3–7	Scripts, store, dashboard, NLP
Deploy	Week 2	Schedule + alerting + docs
Scale	Month 1+	Postgres, hourly, HubSpot

41. Profit & Loss Ownership

- **Owner of P&L impact:** Patrick (GTM AI & Automation). Value is measured as pipeline protected + hours saved; cost is minimal (API + Cowork compute, no new SaaS for v1).
- **Cost line:** negligible infra for Phase 1; Postgres hosting in Phase 2 (to be estimated with infra).

42. Credit / Fault Accountability

- **Credit for success:** Patrick (owner/builder) and the AI GTM team (adoption); Claude/Cowork as the build tool.
 - **Fault for failure:** owned by Patrick as project owner, with clear pre-agreed guardrails (read-only API, no destructive calls, human approval before scaling) so risk is contained. RACI: **R** Patrick, **A** Patrick, **C** RevOps/IT, **I** ELT.
-

Appendix — n8n API Extraction Notes (grounding, no hallucination)

- **List executions:** GET /api/v1/executions — params: status (success|error|waiting), workflowId, projectId, includeData (bool), limit (≤ 250), cursor (pagination). Returns { data: [...], nextCursor }.
- **Single execution (full):** GET /api/v1/executions/{id}?includeData=true — returns execution with data.resultData.runData keyed by node name (each run has startTime, executionTime, data, optional error), data.resultData.lastNodeExecuted (the stop node), and data.resultData.error (top-level error with message, node, stack).
- **List workflows:** GET /api/v1/workflows — params: active (bool), name, tags, limit, cursor. Used to get every workflow's ID + name + active state.
- **Auth:** header X-N8N-API-KEY. Key minted in n8n Settings → n8n API.
- **Rate limits:** paginate with cursor; cap limit at 250; backoff on 429.
- **Extraction rule:** store exactly what the API returns; if a field is absent, record null — never synthesize values.